

OneTrust Cookie-Consent Banner Integration

Documents how `TMHCC_DXLibrary_ClaimForm` loads the OneTrust cookie-consent banner on load.

Files involved

File	Role
<code>index.tsx</code>	Fires the integration once, on mount
<code>utils/oneTrustUtils.ts</code>	All OneTrust logic: fetch config, decide, inject scripts
<code>localizations.json</code>	Supplies the Pega data page name
<code>sections/OneTrustCookieList.tsx</code>	Unused placeholder – see below

Trigger point

`index.tsx` (~line 186-191):

```
// OneTrust cookie-consent banner – gated on this
// environment's Pega Access Group Production Level
// (2 = Dev, 4 = SIT/UAT, 5 = Prod). Declared first so
// it fires before the list-values/hero-image loads below.
useEffect(() => {
  void initializeOneTrustConsent(
    String(1['ProductionLevelDataPageName'])
  );
}, [1['ProductionLevelDataPageName']]);
```

- Runs in a `useEffect` that fires on mount (the dependency is a static localization value, so in practice this runs once).
- It's the *first* effect in the component, ahead of list-values and hero-image loading, so the consent banner has the best chance of appearing before other async work on the page.
- The call is fire-and-forget (`void initializeOneTrustConsent(...)`) – nothing in `index.tsx` awaits it, tracks its success, or reflects its

state in the UI. There is no loading indicator and no error state tied to this call.

What `initializeOneTrustConsent` does

Defined in `utils/oneTrustUtils.ts`. Three steps, in order:

1. Fetch config from a Pega data page

```
async function fetchOneTrustConfig(dataPageName: string) {
  const response = await PCore.getDataPageUtils().getPageDataAsync(
    dataPageName,
    '',
    { AppName: getApplicationName() }
  );
  const enableNotifications =
    response?.pyEnableNotifications === true ||
    response?.pyEnableNotifications === 'true';
  return {
    contentId: response?.pyContentID ?? '',
    enableNotifications
  };
}
```

- The data page name is **not hardcoded** – it comes from `localizations.json`'s `ProductionLevelDataPageName` key, currently set to `"D_EnvironmentConfig"`.
- `AppName` is passed as a data page parameter, resolved via `PCore.getEnvironmentInfo().getApplicationName()`. If `PCore` or that call is unavailable for any reason, `getApplicationName()` swallows the error and falls back to an empty string rather than throwing.
- The data page is expected to return two fields:
 - `pyContentID` – the OneTrust domain/script ID for this environment (what OneTrust calls the "Domain Script ID"). This is what ties the banner to the correct OneTrust project/domain.
 - `pyEnableNotifications` – a flag (accepted as either the boolean `true` or the string `"true"`) that decides whether the banner should actually load in this environment.

- This is the mechanism behind the "Access Group Production Level (2 = Dev, 4 = SIT/UAT, 5 = Prod)" comment in `index.tsx`: the Pega data page itself resolves the current environment/Access Group and returns whether notifications should be enabled – the React code never inspects the raw production-level number, only the boolean it's been reduced to.
- **Failure is silent by design**: the whole fetch is wrapped in try/catch; any failure (data page missing, PCore not ready, network error) returns `null` from `fetchOneTrustConfig`, and `initializeOneTrustConsent` simply does nothing further. No error is logged, thrown, or surfaced to the user – the claim form continues to function normally without a cookie banner.

2. Gate on the response

```
export async function initializeOneTrustConsent(
  productionLevelDataPageName: string
) {
  const config = await fetchOneTrustConfig(
    productionLevelDataPageName
  );
  if (config?.enableNotifications && config.contentId) {
    injectOneTrustScripts(config.contentId);
  }
}
```

Scripts are only injected when **both** are true:

- `enableNotifications` resolved to `true`, **and**
- `contentId` is a non-empty string.

If either is missing, nothing is injected – no banner, no OneTrust SDK on the page at all.

3. Inject OneTrust's scripts

```
function injectOneTrustScripts(domainScriptId: string) {
  if (
    !domainScriptId ||
    document.getElementById(OT_SCRIPT_CONTAINER_ID)
  ) {
```

```

    return;
}

const container = document.createElement('div');
container.id = OT_SCRIPT_CONTAINER_ID;
container.style.display = 'none';

// Appended in order (see notes below):
// 1. OtAutoBlock.js    - blocks cookies pending consent
// 2. otSDKStub.js      - the OneTrust SDK itself
// 3. OptanonWrapper() - required no-op callback stub

document.head.appendChild(container);
}

```

- A hidden(`display: none`) `<div id="tmhcc-onetrust-consent-scripts">` is appended to `document.head`, holding three `<script>` tags:

1. `OtAutoBlock.js`, loaded from

`https://cdn.cookielaw.org/consent/{domainScriptId}/OtAutoBlock.js`

– OneTrust's auto-blocking script, which prevents other cookies/trackers from firing before the user has given consent.

2. `otSDKStub.js`, loaded from

`https://cdn.cookielaw.org/scripttemplates/otSDKStub.js` – the

OneTrust SDK stub itself, with `charset="UTF-8"` and a

`data-domain-script` attribute set to the resolved `domainScriptId`.

This is the script that actually renders the visible cookie banner.

3. An inline script defining `function OptanonWrapper() {}` – an empty

stub. OneTrust's SDK calls `OptanonWrapper()` after it initializes;

without this function existing, the SDK would throw. It's

intentionally a no-op here since the form doesn't need to react to consent changes.

- **Execution order matters and is protected explicitly:** all three scripts have `async = false`. The code comments call this out – script elements created via `document.createElement` default to `async = true`, which would let the browser fetch/execute them out of order. Forcing `async = false` preserves document order (`OtAutoBlock` → SDK stub → wrapper stub), which OneTrust requires.
- **Idempotency guard:** `injectOneTrustScripts` first checks `document.getElementById(OT_SCRIPT_CONTAINER_ID)` and bails out if the container already exists. This means the scripts are only ever

injected once per page load, even if the effect re-runs (e.g. due to a localization value changing) or the component were to remount.

Lifecycle notes

- **No cleanup.** The `useEffect` in `index.tsx` has no cleanup/return function, and `injectOneTrustScripts` never removes its `<div>`. Once injected, the OneTrust scripts and banner persist in the page for the rest of the browser session/page load, even if the claim form component unmounts. This is intentional given OneTrust's own script is meant to be a page-level, one-time bootstrap — but it does mean the banner is not tied to this component's mount/unmount cycle.
- **No retry.** If the data page call fails (e.g. transient network issue), there is no retry logic — the banner simply never appears for that page load.
- **No test/mock coverage.** There is no mock for `D_EnvironmentConfig` in `mock.ts`, and no Storybook story or Jest test exercises this path. In Storybook/local dev without a real Pega runtime, `PCore.getDataPageUtils()` won't exist, `fetchOneTrustConfig` throws, the catch returns `null`, and the banner silently never loads — this is expected, not a bug, but worth knowing when testing locally.

Configuration surface

The only thing intended to change per environment/deployment is the data page name itself, via `localizations.json`:

```
"ProductionLevelDataPageName": "D_EnvironmentConfig"
```

Everything else (the actual domain script ID, the enable/disable flag) is resolved server-side by that Pega data page — this component has no other OneTrust-specific configuration of its own.

Related but inactive piece:

`OneTrustCookieList.tsx`

```
// Drop this on a dedicated Cookie Policy page/section –
// OneTrust renders the generated cookie list into it.
// Not used inside the claim form flow itself, which has
// no such page today.
function OneTrustCookieList() {
  return <div id='ot-sdk-cookie-policy'>&nbsp;</div>;
}
```

- This renders the specific `id="ot-sdk-cookie-policy"` div that OneTrust's SDK looks for to auto-populate a detailed, itemized cookie list (the "what cookies do you actually set" table, distinct from the consent banner itself).
- It is **not imported or rendered anywhere** in this codebase today – confirmed via repo-wide search. It exists as a ready-to-use placeholder for whenever a dedicated Cookie Policy page is built, but currently has no effect on the claim form.
- This is the component referenced in the earlier discussion about adding a "Web Cookie Policy" header link – if that link is ever built to show an in-app cookie list (rather than opening OneTrust's own hosted page), `OneTrustCookieList` is the piece that would need to be wired in.

Quick reference: full flow

- `index.tsx` mounts.
- The mount effect calls `initializeOneTrustConsent('D_EnvironmentConfig')`.
 - This calls `fetchOneTrustConfig('D_EnvironmentConfig')`, which in turn calls `PCore.getDataPageUtils().getPageDataAsync(...)`.
 - On success: returns `{ contentId, enableNotifications }`, read from `pyContentID` / `pyEnableNotifications` on the response.
 - On failure: returns `null` – silent, no banner.
 - If `enableNotifications` and `contentId` are both truthy, calls `injectOneTrustScripts(contentId)`.
 - Appends `<div id="tmhcc-onetrust-consent-scripts">` to `<head>`, containing, in order:
 1. `OtAutoBlock.js` – blocks cookies pending consent.
 2. `otSDKStub.js` – renders the actual banner, tagged with `contentId`.

3. The `OptanonWrapper` stub – required no-op callback.